DRILLER at GenSIE 2026: Interpreting Extraction Semantics through Schema-Aware Prompting, Complementary Retrieval, and Heterogeneous Self-Consistency

Ariel González-Gómez[1], Daniel Alejandro Valdés-Pérez[1]

[1]University of Havana, Havana, Cuba

Abstract

DRILLER is the system submitted to GenSIE 2026 for Spanish schema-guided information extraction. GenSIE asks systems to extract grounded JSON objects from Spanish texts under JSON Schemas supplied at inference time. The main challenge is not only JSON validity, but field interpretation: names, descriptions, types, enum labels, nullability, and nesting can change from one instance to the next. DRILLER addresses this setting with three submitted pipelines built on a common schema-guided extraction architecture. `Schema-RAG` renders schemas in a model-readable form, constrains generation with a strict structured response format, and adds field-aware retrieved demonstrations. `Reasoned-RAG` adds temporary top-level reasoning/value wrappers so the model can assess local evidence before producing each final field value. `Mixed-SC` combines direct and reasoning-augmented trials, then selects a final prediction through schema-aware aggregation. Across the pipelines, DRILLER emphasizes schema interpretation, grounded extraction, conservative postprocessing, and compatibility with the model-agnostic hosted evaluation protocol.

Keywords

Spanish information extraction, schema-guided extraction, retrieval-augmented generation, structured generation, self-consistency

1. Introduction

Schema-guided information extraction requires systems to adapt to an output contract that may be unknown before inference time. Instead of extracting a fixed inventory of slots, the model receives a task-specific schema and must decide what each field means for the current source text. This setting is increasingly common when language models are used behind APIs, form-filling workflows, or data integration tools: the schema is not just a validator, but part of the instruction. [1]

GenSIE 2026 formalizes this problem for Spanish information extraction [2]. Each instance provides a Spanish source text, an extraction instruction, and a target JSON Schema; the system must return a grounded JSON object that conforms to that schema. The task is part of IberLEF 2026 [3] and uses a hosted, model-agnostic evaluation protocol, so a submitted system must respect the model selected by the evaluator rather than relying on a privately chosen backend. This makes inference-time methods such as schema presentation, retrieval, constrained generation, and aggregation central to the submission.

JSON validity is only one part of the task. A model can return a syntactically valid object and still fail because it has misread the schema. A string field may require a copied mention, a normalized value, or a short grounded summary depending on its description. An enum may require mapping textual evidence to labels that do not appear literally. A nullable field may invite a plausible answer from world knowledge even when the passage does not support it. Arrays and nested objects add ambiguity about completeness and item identity. The model must align each value with the current field semantics, not only with the JSON shape.

Variable schemas amplify this field-interpretation problem. With a fixed schema, field semantics could be learned from training data or encoded in task-specific rules. In GenSIE, field names, descriptions, type

IberLEF 2026: Iberian Languages Evaluation Forum, 2026

constraints, enum labels, nullability, and nesting are runtime evidence. DRILLER uses them to decide which text spans are relevant, when normalization is expected, when absence should be represented as JSON `null`, and how much structure to return.

DRILLER, originally named for *Dynamic Reasoning for Information Labeling and Literal Evidence Retrieval*, was submitted to the challenge as three pipelines. We refer to:

- `enriched-schema-rag` as `Schema-RAG`, the direct retrieval-augmented extractor.
- `enriched-inline-reasoning-rag` as `Reasoned-RAG`, which uses the same retrieval path but temporarily wraps top-level fields as reasoning/value objects during generation.
- `mixed-extractors-self-consistency-rag` as `Mixed-SC`, which combines direct and reasoning-augmented trials through heterogeneous self-consistency and schema-aware aggregation.

These aliases are used throughout the paper.

All three pipelines share a schema-guided extraction spine. DRILLER renders the target schema in a compact Pydantic-style view for the prompt, sends a strict JSON Schema response format to constrain generation, retrieves up to two field-aware demonstrations from a curated few-shot corpus, and applies deterministic postprocessing before returning the final object. The pipelines differ in whether they use a temporary reasoning scaffold and whether they aggregate multiple candidates.

The readable schema presented in the prompt foregrounds field names, descriptions, types, enums, nullability, and nesting. The generation schema constrains the model to return a complete JSON object. This separation lets DRILLER present the schema in a typed, class-like form while preserving the evaluator's original JSON shape.

Field-aware few-shot retrieval selects examples by schema relevance. DRILLER first looks for demonstrations with the same normalized schema. When exact structural matches are unavailable, it falls back to field-level retrieval with type-compatible matching and a complementary pair objective. Retrieved examples illustrate extraction behavior, including contrasts such as present versus absent values, but their content is not evidence for the current instance.

`Reasoned-RAG` adds an internal reasoning interface, useful for fields whose values require evidence assessment, null decisions, or schema-specific label mapping. During generation, each top-level field is represented as an object with a `reasoning` string and a `value` of the original field type. After generation, the reasoning strings are discarded and only the values are returned. `Mixed-SC` then uses both direct and reasoned extractors as heterogeneous candidate generators, aggregating their final-shape JSON outputs with recursive schema-aware rules for scalars, nulls, objects, and arrays.

The contributions of this work are:

- a modular schema-guided extraction architecture for variable JSON Schemas in Spanish information extraction;
- schema-readable prompting paired with strict structured generation, separating schema interpretation from output-interface enforcement;
- field-aware few-shot retrieval with same-schema matching, type-compatible field fallback, and complementary example selection;
- a temporary top-level reasoning/value transformation that supports field-local evidence assessment while preserving the final schema interface;
- heterogeneous self-consistency over direct and reasoning-augmented RAG extractors with conservative schema-aware postprocessing and aggregation.

2. Task Setting

GenSIE is a Spanish schema-guided information extraction task [2]. Each instance provides a source text, a natural-language extraction instruction, and a target JSON Schema. The source text is the only evidence for the answer. The instruction states the extraction goal, and the schema defines the output

Table 1
Submitted DRILLER pipelines.

Alias	Schema view	Reasoning wrapper	Requested trials	Final output
`Schema-RAG`	Pydantic	No	1	Direct
`Reasoned-RAG`	Reasoned Pydantic	Top-level	1	Unwrap + direct
`Mixed-SC`	Direct and reasoned	Reasoned trials	up to 4	Schema-aware SC

structure through field names, descriptions, primitive types, objects, arrays, enum labels, nullability, and required properties.

The required output is a JSON object grounded in the source text and conforming to the target schema. Conformance includes producing the expected object shape, using valid primitive and enum values, and representing nested structures and arrays according to the schema. Grounding means that values should be supported by the current passage rather than external knowledge, retrieved examples, or plausible background assumptions.

Schemas vary across instances, so the extractor cannot rely on a fixed ontology or stable slot inventory. The same textual mention may be relevant for one schema and irrelevant for another; the same JSON type may correspond to different behaviors depending on the field description. The schema must therefore be interpreted dynamically at inference time.

The official protocol also constrains inference cost. The submission guidelines describe token and wall-clock budgets as soft averages over the 100 test instances: a target of 32K total input-plus-output tokens per instance, including reasoning, retries, and self-corrections, and a target of 60 seconds of user-code time per instance [4]. These constraints mean that a system cannot assume arbitrarily many model calls, unbounded retries, or unlimited reasoning traces; inference-time methods must be organized as a bounded procedure.

The task description states that the private evaluation set contains 100 new instances split roughly evenly between schemas that appear in the development set and schemas that do not [5]. This overlap does not reuse the development source documents, but it makes exact or near-exact schema reuse a realistic evaluation condition. DRILLER's same-schema retrieval is therefore a deliberate inference-time strategy: it first searches for structurally identical schema demonstrations before falling back to field-level analogies when no suitable same-schema references are available.

Missing information must be handled conservatively. When the requested value is not supported by the source text and the schema permits null, the system should return JSON `null`. If evidence is present, returning `null` is also an error. DRILLER therefore treats nullability as a schema-guided evidence decision rather than a generic fallback.

Official dataset construction, scoring, ranking, and evaluation protocol details are specified in the GenSIE overview paper. We focus on the submitted DRILLER pipelines and their inference-time mechanisms for producing grounded schema-conformant JSON under that protocol.

3. System Overview

DRILLER submitted three pipelines: `Schema-RAG`, `Reasoned-RAG`, and `Mixed-SC`. They share a schema-guided extraction architecture and differ in reasoning scaffolds, trial count, and final decision rule. Their submitted interface names are `enriched-schema-rag`, `enriched-inline-reasoning-rag`, and `mixed-extractors-self-consistency-rag`. Non-registered experimental variants are outside the submitted system.

Table 1 summarizes the final pipelines. All three use retrieval-augmented prompting and strict structured JSON generation.

Figure 1 shows the shared path from a GenSIE instance to a final JSON object. The input supplies the Spanish source text, extraction instruction, and target JSON Schema. DRILLER renders the schema for prompting, retrieves local demonstrations, constructs a Spanish extraction prompt, requests a structured

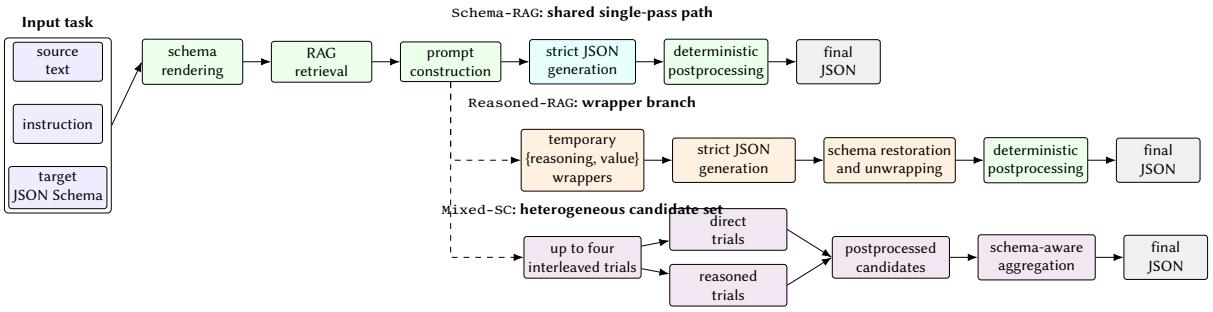

Figure 1: Architecture of the submitted DRILLER pipelines. `Schema-RAG` follows the shared single-pass path, `Reasoned-RAG` adds temporary reasoning/value wrappers before restoring the original schema, and `Mixed-SC` aggregates postprocessed candidates from direct and reasoned trials.

JSON object from the hosted model, parses and normalizes the output, and optionally aggregates multiple candidates. The returned object always has the original challenge schema shape.

The prompt presents the instance instruction, grounding rules, retrieved examples when available, a Pydantic version of the target schema, and the source text. The rules state that the source text is the only evidence, examples are demonstrations rather than facts for the new instance, enum labels must be used exactly, and unsupported nullable fields should be returned as JSON `null`. This organization keeps the schema and passage close to generation while making the evidence boundary explicit.

Schema rendering has direct and reasoned variants. In direct mode, the JSON Schema is shown as a compact Pydantic-style model: fields become typed attributes, descriptions remain near the fields they define, arrays and nested objects are readable as typed structures, and enums are displayed as literal alternatives. This prompt view is not the formal output contract. The model request also includes a strict JSON Schema response format derived from the target schema, and the final answer is checked against the evaluator-facing schema shape after postprocessing.

Retrieval is also shared. Each extraction can include up to two local few-shot demonstrations selected for schema relevance. Same-schema retrieval is used when examples have the same normalized structural schema as the current task; otherwise, DRILLER falls back to field-level matching over field names, descriptions, type classes, and complementary coverage. The selected examples are rendered in the same output style as the current extractor: direct examples for `Schema-RAG`, reasoning/value examples for `Reasoned-RAG`, and trial-specific rendering inside `Mixed-SC`.

Postprocessing is deterministic and conservative. It parses JSON, cleans common Unicode artifacts, normalizes strings to *Normalization Form C* (NFC), removes temporary reasoning wrappers when present, and converts the literal string `"null"` to JSON `null` only at schema positions where null is allowed. It does not infer missing facts, repair unsupported values, or reinterpret the source text after generation.

`Schema-RAG` is the direct single-pass pipeline. It uses the Pydantic-style prompt schema, retrieves demonstrations, requests a strict JSON object, and returns the parsed and normalized object. Because no reasoning wrapper is used, the generated top-level fields already match the final output interface.

`Reasoned-RAG` follows the same path but temporarily transforms each top-level field into an object with `reasoning` and `value` slots during generation. The prompt schema exposes this as a reasoned value, and the response schema requires the wrapper structure. After generation, DRILLER discards the reasoning strings and returns only the values in the original schema shape.

`Mixed-SC` moves from a single extraction to a heterogeneous candidate set. It requests up to four budget-aware trials: two direct `Schema-RAG`-style attempts and two reasoning-augmented `Reasoned-RAG`-style attempts in an interleaved plan. Each completed trial performs its own retrieval, prompt construction, strict generation, and postprocessing. Aggregation receives final-shape JSON candidates, not raw model messages or reasoning traces, and combines them recursively using schema-aware rules for scalars, nulls, objects, and arrays.

Table 2
Schema rendering footprint over 13 unique development schemas. Values are estimated prompt tokens computed as $\lceil \text{characters}/4 \rceil$.

Rendering	Mean	Median	Min	Max	Mean vs. raw
Raw JSON Schema	570	660	109	886	–
Direct Pydantic view	276	310	67	385	-51.6%
Plain compact Pydantic renderer	249	283	41	357	-56.3%
Reasoned Pydantic view	272	306	58	378	-52.2%

4. Schema-Guided Prompting

DRILLER treats the schema as part of the task semantics, not only as a formatting constraint. The prompt helps the model decide what each field asks for, while the model request constrains the response to a JSON object.

The extraction prompt asks the model to use the current source text as the only evidence, follow the extraction instruction, and interpret field meanings through the target schema. Retrieved examples are demonstrations of extraction behavior, not evidence for the current input. The answer must be only the requested JSON object, without markdown fences, explanations, or extra text.

The standard prompt sets the role of a precise Spanish information extractor, gives rules for grounding, full schema coverage, exact enum labels, conservative null handling, and requested normalization, then inserts retrieved examples. It renders the current schema in a model-readable form immediately before the source text, keeping the schema and evidence close to generation.

Raw JSON Schema is well suited for validation, but it can be visually heavy inside a prompt. Nested property dictionaries, separate required lists, repeated keywords, and nullable type arrays can obscure the field descriptions that carry extraction semantics. DRILLER therefore renders the schema in a Pydantic-style view. The view is not an executable program and does not replace the target JSON Schema; it is a compact and more readable representation of the current extraction contract.

In this view, object schemas become class-like structures and properties appear as typed attributes. Primitive fields are rendered as familiar scalar types, arrays as list types, enums as literal alternatives, and nested objects as nested or referenced model types. Nullability is shown near the field rather than separated from the definition. Field descriptions are preserved because they often determine the correct behavior: the same type annotation can require copying, normalization, classification, or a short grounded summary depending on the description.

Figure 2 shows the transformation on a compact excerpt from the `cultural_media_001` development schema. The raw JSON Schema excerpt carries the validation-oriented structure: a definition for the enum, property dictionaries, and nullable alternatives. The right panel shows two Pydantic-style renderings for the same selected fields: the plain compact renderer and the reasoned view. Both define the prompt-only alias `Nullable[T] = T | null`; the reasoned view additionally defines the generic `Reasoned[T]` wrapper and changes each root field from T to `Reasoned[T]`. Nested structures remain inside the `value` slot rather than being wrapped recursively.

We measured the schema-text footprint for the 13 unique target-schema structures observed in the development set, deduplicated by canonicalized `target_schema` JSON. The raw baseline is the indented JSON Schema text produced by the prompt schema renderer. The direct and top-level-reasoned Pydantic rows use the actual schema-view code paths selected by `SchemaPromptMode.PYDANTIC` and `SchemaPromptMode.REASONED_PYDANTIC`. We also report `render_plain_pydantic_schema`, the same compact prompt-only renderer family used by the reasoned view, but without `Reasoned[T]` wrappers; this row isolates wrapper overhead from unrelated prelude differences. We estimate size as $\lceil \text{characters}/4 \rceil$, matching the character-based token fallback used by DRILLER's trial-budget planner. The measurement estimates prompt footprint only, not model-specific tokenization.

Table 2 describes schema text alone, not the complete prompt or completion cost. On these schemas, the class-like view roughly halves the prompt-side schema footprint relative to indented raw JSON

Raw JSON Schema view

```
{
  "$defs": {
    "MediumType": {
      "enum": [
        "MOVIE",
        "BOOK",
        "TV_SERIES",
        "VIDEO_GAME"
      ],
      "title": "MediumType",
      "type": "string"
    }
  },
  "properties": {
    "medium": {
      "$ref": "#/$defs/MediumType",
      "description": "The type of media being
     reviewed"
    },
    "director": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "description": "The person who directed
     the film or series",
      "title": "Director"
    }
  }
}
```

Plain compact Pydantic renderer

```
Nullable[T] = T | null

MediumType = Literal["MOVIE", "BOOK", "
     TV_SERIES", "VIDEO_GAME"]

class Output(BaseModel):
    medium: MediumType = Field(..., description
     ="The type of media being reviewed")
    director: Nullable[str] = Field(None,
     description="The person who directed the
     film or series")
```

Reasoned Pydantic view

```
Nullable[T] = T | null

class Reasoned[T](BaseModel):
    reasoning: str
    value: T

MediumType = Literal["MOVIE", "BOOK", "
     TV_SERIES", "VIDEO_GAME"]

class Output(BaseModel):
    medium: Reasoned[MediumType] = Field(
     description="The type of media being
     reviewed")
    director: Reasoned[Nullable[str]] = Field(
     description="The person who directed the
     film or series")
```

Figure 2: Schema transition example from a reduced `cultural_media_001`. The left panel shows a raw JSON Schema excerpt from the task's `target_schema`; the right panel shows the corresponding selected-field excerpts emitted by the plain compact and reasoned Pydantic-style prompt renderers. The excerpt keeps a string-valued enum field, a nullable field, and field descriptions while omitting root metadata and unrelated fields for space.

Schema. The actual reasoned view is slightly shorter than the actual direct Pydantic view because the direct code path includes import-style boilerplate, while the reasoned prompt renderer uses a prompt-only prelude. In the like-for-like renderer comparison, top-level reasoning wrappers increase the mean footprint from 249 to 272 estimated tokens. The reasoned view remains in the same range because the submitted reasoning mode wraps only top-level fields.

DRILLER uses three schema roles, summarized in Table 3. The same target schema is transformed differently when shown to the model, used to constrain decoding, or returned to the evaluator.

The prompt schema is a readability layer. It can use class-like declarations, readable nullable notation, and other prompt-only conventions because it communicates the schema to the model. The generation schema has a different role: DRILLER attaches a strict JSON Schema response format to the model request so decoding is constrained toward a JSON object compatible with the requested structure.

For the submitted non-baseline pipelines, declared object properties are required during generation. This reduces field omissions, a common failure mode when a model understands the task but leaves out keys. Requiring keys does not require every value to be substantive. If the target schema permits null, the generated object may include the key with JSON `null`. The final challenge schema remains the original schema shape; generation-time requiredness is an inference constraint, not permission to fabricate unsupported content.

Reasoning wrappers create a second schema variant. In direct mode, used by `Schema-RAG`, the prompt schema is ordinary Pydantic-style rendering and the model produces final values directly.

Table 3
Schema roles in DRILLER. The prompt schema supports model comprehension; the generation schema constrains decoding; the final challenge schema defines the returned interface.

Schema role	Where it is used	Function in the system
Prompt schema	Text shown inside the extraction prompt	Pydantic-style or reasoned Pydantic-style rendering that helps the model interpret field names, descriptions, types, enums, nullability, and nesting.
Generation schema	Structured response format sent with the model request	Strict JSON Schema used to constrain the generated object; in reasoning mode, temporary wrappers are inserted before generation.
Final challenge schema	Shape expected by the evaluator	Original task schema after schema restoration when needed and deterministic cleanup; reasoning scaffolds and prompt-only conventions are not returned.

In reasoning mode, used by `Reasoned-RAG` and the reasoning-augmented trials of `Mixed-SC`, each top-level field is shown as a reasoned value. The temporary generation schema requires an object with `reasoning` and `value` fields for each top-level slot. The `value` field retains the original type, including enums, arrays, objects, and nullable alternatives.

The reasoning transformation changes the temporary interface without asking for free-form explanation. The model has a structured place to state local evidence or absence before committing to a field value, and schema restoration removes the reasoning text before returning the final object. Direct and reasoned prompting therefore share the same final objective while exposing different intermediate interfaces.

Prompting also interacts with retrieval. In the general RAG layout, each example carries enough schema context for the model to understand why its output follows from its own source text and schema. In the same-schema layout, selected examples share the normalized target schema, so DRILLER can render the shared schema once and show compact example inputs and outputs. This saves prompt size and lets the examples emphasize contrasting field outcomes under the same structure.

Examples can show how fields are populated, when null or empty arrays are appropriate, and how enum labels or nested structures behave, but their values are not evidence for the current instance. In reasoning mode, retrieved examples are rendered with the same temporary `{reasoning, value}` structure expected from the current extractor; in direct mode, examples use final field values. `Mixed-SC` applies the appropriate layout separately inside each trial.

Schema-guided prompting in DRILLER combines readable schema presentation, explicit grounded-extraction rules, strict structured generation, and retrieval-aware layout. It makes schema interpretation explicit while returning a complete JSON object compatible with the evaluator-facing schema.

5. Field-Aware Few-Shot Retrieval

DRILLER retrieves examples for schema interpretation. In a variable-schema task, semantic similarity between source texts is not enough: two passages may be topically close but ask for different fields, while two different domains may instantiate similar extraction behavior. The submitted pipelines retrieve demonstrations using schema structure, extraction instruction, field descriptions, type compatibility, and complementary output behavior. `Schema-RAG`, `Reasoned-RAG`, and every trial of `Mixed-SC` use the same retrieval path.

5.1. Curated Demonstration Corpus

The retrieval pool is a local few-shot prompting (FSP) corpus of structured extraction demonstrations. Each case contains a Spanish source text, an extraction instruction, a target schema, expected values, tags describing covered extraction phenomena, and field-level rationales or descriptions that support rendering examples in either direct or reasoning-augmented form. The corpus is synthetic and curated rather than mined from an external collection.

The GenSIE development set guided corpus design. We used it to study schema structure, source-text style, vocabulary, noise patterns, and recurring ambiguities, rather than reusing development instances as demonstrations. We also manually curated a small development-time evaluation subset, reviewing up to two instances per task family. Those curated instances served as design references only. Direct reuse would not provide systematic complementary coverage of field dualities and would contaminate evaluation on the same curated subset, since retrieval could return the instance being solved.

We constructed the final FSP corpus as a compact library of synthetic and manually curated structured cases, organized by development task family and schema family. The corpus used an author-in-the-loop workflow assisted by an agentic LLM coding assistant. The assistant helped inspect candidate task families, draft complementary-case proposals, generate provisional source texts and structured resources, and run structural checks. The authors reviewed and corrected the proposed dualities, source texts, expected outputs, reasoning fields, enriched descriptions, and final JSON resources before inclusion. The workflow was:

1. Each development task family became the unit of work because a family bundles a schema, domain, source-text style, vocabulary, and recurring extraction traps.
2. The assistant inspected the curated development subset first, then consulted the full uncurated development set when more evidence about style, structure, or ambiguity was needed. The full set characterized text length, noise patterns, and extraction behavior, but was not treated as trusted gold output.
3. The assistant identified useful field-level dualities: nullable value versus grounded non-null value; empty array versus populated array; boolean `true` versus `false`; frequent enum label versus an alternative label; direct or near-verbatim text versus grounded summary; explicit value versus distractor or insufficient evidence; and normalized value versus literal mention.
4. It then proposed two complementary cases for the task or schema family, using a balanced or approximately balanced mask over dual fields as a design heuristic. One case covered one side of several dualities and the second covered the inverse side; qualitative contrasts such as enum A versus enum B or explicit date versus nearby distractor date were handled outside a rigid binary mask.
5. The authors reviewed this pre-drafting proposal, including selected references, field dualities, complementary mask, and intended case behavior. Full synthetic drafting began only after this gate was accepted.
6. Under the approved design, the assistant drafted synthetic Spanish source texts, candidate expected outputs, and field-level reasoning. The texts had to ground values, `null`s, and empty arrays naturally, resemble the reviewed family style, and avoid benchmark-aware phrases that mechanically announced the expected answer or absence.
7. The authors manually curated the drafted cases before final resource generation, editing source texts, expected outputs, and field-level reasoning. This review checked that each value, `null`, and empty list was justified by the source text and that the pair contrasted the intended regimes.
8. After author approval, the assistant generated the final `StructuredFspCase` JSON resources, including `field_examples`, tags, field-level reasoning, and enriched descriptions when applicable. The resources were loaded through the FSP resource loader or equivalent structural checks before retrieval used them.

The final extraction corpus contains two cases for each of the 22 schema or task families represented in the curated development setting, for 44 extraction cases in the local retrieval pool. The families cover

Table 4
Complementary FSP pair for the `technical_software` schema family.

Field or contrast	*Lince Editor* case	*QuantaPack Pro* case
`official_name` and `app_category`	Direct product name; enum maps to `IDE_EDITOR`.	Abbreviated heading must resolve to full name; enum maps to `ARCHIVER`.
`platforms` and `features`	Populated arrays with named operating systems and technical capabilities.	Empty arrays: installers and downloads are mentioned, but no supported OS or concrete feature list.
`exact_release_date`	`null`: only year-level or relative release evidence is present.	Exact first-release date normalized to `DD/MM/YYYY`.
`latest_stable_version_sha256`	`null`: PGP signatures and a nightly commit hash are distractors.	Explicit SHA256 for the stable installer.
`current_ceo_name`	`null`: founder, idea author, and coordinating team are not CEO evidence.	Explicit current CEO of the developing company.
Pair lesson	Extract grounded scalar values and populated arrays, but abstain for nullable fields with only distractor evidence.	Resolve aliases, keep unsupported arrays empty, and fill nullable fields only when the source states the required evidence.

a range of domains and structures, but the corpus is not intended to be a large topical nearest-neighbor database. It is a compact library of contrasted demonstrations: the useful unit is often a pair of cases showing how the same or related fields behave under different evidence conditions.

The paired organization is deliberate. A nullable field benefits from both grounded non-null values and grounded absence. An array field benefits from examples where the correct list is populated and examples where it is empty. Enum fields benefit from contrasting labels, especially when labels are inferred rather than copied literally. Numeric and date fields may require exact copying in one case and normalization in another. String fields may require direct mention extraction, near-verbatim evidence, or a short grounded summary depending on the field description.

After complementary cases had been proposed and reviewed, we added enriched field descriptions as compact Spanish prompt annotations. They explain how to extract a field and warn about traps observed during development-example review or synthetic case construction, such as distractor dates, nearby but semantically wrong entities, aliases, or conditions under which a nullable field should remain `null`. These descriptions clarify schema semantics for same-schema prompting; they do not add new instance evidence. They appear only in the prompt-side schema view. The target JSON Schema, strict generation schema, and evaluator-facing output schema remain unchanged.

The FSP reasoning fields follow the structured evidence discipline. In reasoning-mode examples, each field can render `field_asks`, `relevant_fragments`, and `final_value` as a local evidence protocol. Cited fragments support the extraction decision rather than merely repeating the answer string. They stay short while preserving the context needed to justify the value: accepted candidates show why the value belongs to the requested field, and rejected or `null` cases cite the closest reviewed context and state which required assertion is missing. Enum reasoning considers the allowed labels and justifies the selected label from relevant fragments. Array reasoning may cite a complete list once, or cite compactly around individual items, so examples support recall without bloating the prompt. FSP examples therefore demonstrate grounding behavior as well as output formatting.

Table 4 gives one compact example from the `technical_software` family. The two cases share the same schema but expose complementary regimes: one case grounds populated arrays and leaves several adversarial nullable fields unsupported, while the other inverts the pattern with empty arrays, alias resolution, exact date normalization, an explicit stable-installer checksum, and explicit corporate metadata.

Coverage includes null versus non-null values, present versus absent evidence, enum alternatives, empty versus populated arrays, arrays of simple values, arrays of objects, nested objects, direct strings, summary strings, boolean inference, numeric normalization, date normalization, bounded scores, long evidence spans, and distractor mentions. These contrasts matter because few-shot examples can bias the model when they show only one outcome regime. Complementary demonstrations teach conditional

behavior: extract when evidence exists, abstain when it does not, normalize only when requested, and respect the requested type and label set.

5.2. Same-Schema Retrieval

At runtime, DRILLER first tries to retrieve examples with the same normalized schema as the current task. Let S_q be the target schema of query instance q, and let S_c be the schema of a candidate case c. DRILLER computes a normalized schema fingerprint $\phi(S)$ by canonicalizing the JSON representation while preserving the structural extraction contract: field names, object structure, array structure, primitive types, enum alternatives, required properties, and nesting. Descriptions are neutralized so that prompt wording refinements do not change schema identity. Order-insensitive elements such as `required`, `enum`, and array-valued `type` are sorted before serialization. Same-schema candidates satisfy the hard constraint

$$\phi(S_q) = \phi(S_c).$$

The fingerprint separates schema identity from topical similarity. A medical passage and a technical passage might both contain names, dates, arrays, and classifications without sharing an extraction form. Conversely, two source texts from the same schema family may require the same field interpretation even when their values differ. Same-schema retrieval therefore treats structural identity as a filter rather than as a soft ranking feature.

Filtered candidates are then ranked semantically. DRILLER builds a global textual representation $r(q)$ of the current task from the task identifier, extraction instruction, and textual representations of top-level fields. Candidate cases are represented analogously as $r(c)$. These representations are embedded and compared by cosine similarity,

$$\mathrm{sim}(r(q), r(c)).$$

The submitted system uses a same-schema threshold $\tau = 0.6$. If at least two fingerprint-matching candidates satisfy $\mathrm{sim}(r(q), r(c)) \geq \tau$, DRILLER selects the top candidates by similarity and marks them as same-schema demonstrations. If fewer than two candidates pass this test, retrieval continues with the field-level fallback.

Same-schema examples show the exact output contract under different source texts. They clarify what counts as evidence for a field, when null or an empty list is correct, and how enum labels or nested structures are populated. They also enable a compact prompt layout: the shared schema is rendered once, and examples focus on instruction, source text, and expected output rather than repeating identical schema blocks.

Same-schema mode can also enrich the prompt-side schema view with curated field descriptions drawn from the selected cases. These enriched descriptions add task and domain guidance such as enum interpretation, null traps, or distractor patterns to the schema text shown to the model. They do not modify the target JSON Schema, the strict generation schema, or the final evaluator-facing schema.

5.3. Field-Level Retrieval Fallback

When same-schema retrieval does not produce enough candidates, DRILLER falls back to field-level retrieval. Let the query top-level fields be $F_q = \{f_1, \ldots, f_m\}$, in schema order, and let F_c be the top-level fields of candidate case c. Each field f is mapped to a textual representation $t(f)$ that combines its field name, readable type, and description, for example "`name (type): description`". The retriever also assigns a coarse type class $\kappa(f)$, ignoring nullability.

The coarse classes used by the retriever are `string`, `boolean`, `numeric` for both integer and number fields, `enum`, `object`, and recursively typed arrays such as `array[string]`, `array[numeric]`, `array[enum]`, and `array[object]`. A fallback `any` class is available for unsupported or underspecified schemas. We write $\kappa(f_i) \sim \kappa(g)$ for compatibility; in the submitted implementation this relation is equality of the nullability-insensitive coarse classes. Thus nullable and non-nullable variants of the

same base type can match, while an array of objects is not aligned with a scalar summary merely because their descriptions share topical words.

For each candidate c, DRILLER embeds the field texts $e(t(f))$ and computes, for every query field f_i, the best compatible field match in the candidate:

$$a_i(c) = \max\left(0, \max_{g \in F_c,\ \kappa(g) \sim \kappa(f_i)} \cos(e(t(f_i)), e(t(g)))\right),$$

with $a_i(c) = 0$ when no compatible candidate field exists. The candidate is therefore represented by a field-similarity vector aligned with the current query schema,

$$v(c) = (a_1(c), \ldots, a_m(c)).$$

A candidate's individual relevance can be summarized by $\|v(c)\|_2$, but two-example selection uses a pair-level objective over these same vectors.

The type-aware vector prevents topical retrieval from ignoring output shape. It avoids matching semantically similar descriptions with incompatible JSON structures, allows nullable and non-nullable variants of the same base type to reinforce each other, and shows whether a candidate is broadly useful or strong for only part of the schema. Since retrieval selects a pair of demonstrations, two individually imperfect candidates can be preferable when they cover different field behaviors.

If embedding retrieval fails or produces no usable candidates, DRILLER uses a lexical/schema fallback. It scores candidates with explicit features such as exact normalized schema matches, compatible field fingerprints, tag overlap, field-name overlap, shared terms, and domain-term overlap, with a small penalty for long example texts. This fallback preserves schema-aware retrieval when embeddings are unavailable or uninformative.

5.4. Complementary Pair Selection

The submitted RAG path retrieves up to two examples. In the field-level fallback, DRILLER selects a pair with a complementarity-aware objective rather than choosing the two highest individual scores. For two candidate demonstrations c_1 and c_2, with field-similarity vectors $v(c_1)$ and $v(c_2)$ defined above, the pair score is

$$s(c_1, c_2) = \|v(c_1) + v(c_2)\|_2 + \|v(c_1) - v(c_2)\|_2.$$

The first term, $\|v(c_1) + v(c_2)\|_2$, rewards joint relevance and field coverage. If both examples provide useful analogues for the current schema, their combined vector has large magnitude. This term aligns with the ordinary retrieval goal that demonstrations should be relevant to the task.

The second term, $\|v(c_1) - v(c_2)\|_2$, rewards complementarity. If two candidates are strong on the same fields and weak on the same fields, their difference vector is small. If one is strong where the other is weak, the difference grows. This discourages near-duplicate demonstrations that reinforce one behavior while leaving other fields unexplained.

Demonstration diversity is not merely topical diversity. Two examples from different domains can still be redundant if both show easy scalar fields and neither illustrates nulls, arrays, or enum decisions. Conversely, two examples from a related family may be complementary if one illustrates absence and empty arrays while the other illustrates populated nested objects and label mapping. Tie-breaking favors stronger individual relevance and stable resource order, but the core decision is pair-level.

5.5. Prompt Integration

DRILLER integrates selected examples into the extraction prompt in one of two layouts. The general RAG layout is used when examples are related but not all same-schema matches. Each example is rendered as a self-contained demonstration with its instruction, schema context, source text, and

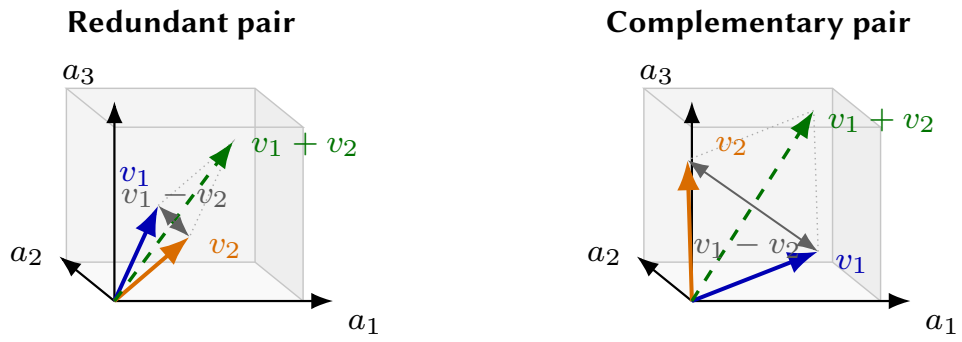

Figure 3: Field-aware pair selection as a vector objective. Each axis is a query-field similarity component a_i; actual retrieval vectors are m-dimensional. A redundant pair concentrates evidence in similar directions, so $v_1 - v_2$ is short. A complementary pair covers different field dimensions, increasing both the joint vector $v_1 + v_2$ and the difference vector $v_1 - v_2$ used by the pair score.

expected output. This gives the model enough context to see how the example output follows from that example's schema before using it analogically.

The same-schema layout is used when all selected examples share the normalized target schema. The shared schema is rendered once, alongside the role instruction, extraction rules, root description when available, and guidance that examples are demonstrations rather than evidence. The examples then omit repeated schema blocks and focus on source-to-output behavior under the common structure. This reduces prompt length and makes contrasting field outcomes more salient.

Pipeline integration depends on the current output interface. In `Schema-RAG`, examples are rendered in direct final-value form, matching the object shape returned after generation. In `Reasoned-RAG`, examples are rendered with top-level `{reasoning, value}` objects because the temporary generation interface requires that structure. The reasoning strings demonstrate a local evidence protocol, while the `value` slots show the values that will later be unwrapped.

`Mixed-SC` uses the same retrieval machinery inside each trial. Direct trials use the `Schema-RAG` rendering, and reasoning-augmented trials use the `Reasoned-RAG` rendering. Each trial performs retrieval independently. Aggregation receives only postprocessed JSON candidates in the final schema shape.

Field-aware retrieval turns the local corpus into an inference-time schema interpretation aid. Same-schema retrieval exploits exact structural matches when available, field-level fallback finds type-compatible analogies, the pair objective favors complementary coverage, and prompt integration adapts examples to the direct, reasoned, and mixed pipelines.

6. Reasoning-Augmented Extraction

`Reasoned-RAG` adds a temporary reasoning interface to the shared extraction pipeline. Some GenSIE fields require local evidence assessment before a value can be produced: the model may need to identify relevant fragments, decide whether evidence is absent, normalize a phrase, or map cues to an enum label. The reasoned interface gives each top-level field a structured place for that intermediate decision while preserving the final JSON interface.

In `Reasoned-RAG`, each top-level field of the target object is temporarily wrapped as an object with two properties:

```
{
  "reasoning": "...",
  "value": ...
}
```

The `reasoning` property is a string. The `value` property keeps the original schema of the wrapped field. If the original field is an enum, the `value` must be one of the enum labels; if it is an array or object, the nested structure remains there; if it is nullable, the `value` may be JSON `null` when the schema permits it. The wrapper changes the temporary generation shape, not the semantic target.

The submitted reasoning mode is top-level only. It wraps root object properties, not every nested property recursively. If a top-level field is itself an object or an array of objects, the nested content remains inside `value` according to the original field schema. This keeps the prompt and response format manageable while asking the model to deliberate at the level of the main requested slots.

In the prompt schema, each top-level field of type T is shown as a `Reasoned[T]` value. The extraction instructions do not ask for arbitrary chain-of-thought. They require each `reasoning` string to use three literal Spanish section labels, in this order:

```
EL CAMPO PIDE: ...
FRAGMENTOS RELEVANTES: ...
VALOR FINAL: ...
```

The first section, EL CAMPO PIDE, makes the model restate the local schema request before filling the slot. This forces interpretation of the field name, type, enum alternatives, nullability, and description, which is especially useful when two fields share the same JSON type but ask for different extraction behavior. The second section, FRAGMENTOS RELEVANTES, bounds the evidence used for the decision. It asks for only the source fragments that decide the field, or an explicit statement of absence when the field is unsupported. This discourages importing evidence from few-shot examples or from general knowledge. The third section, VALOR FINAL, connects the evidence to the schema-compatible value that will be placed in `value`: an exact enum label, a normalized scalar, an array, an object, an empty array, or JSON `null` when permitted.

The reasoning interface is part of the strict temporary response schema, so each wrapped top-level field must contain both `reasoning` and `value`. This keeps reasoning inside the JSON object and keeps the final value constrained by the original field type. The wrapper adds a field-local explanation channel without relaxing the schema constraints on the answer.

After generation, DRILLER restores the original schema. Each top-level object of the form `{reasoning, value}` is replaced by its `value`, returning the object to the evaluator-facing schema. This wrapper removal is the inverse of the temporary schema transformation, not semantic post-hoc repair. The reasoning strings may be retained as trace metadata, but they are discarded before the prediction is returned or passed to self-consistency aggregation. If the expected wrapper structure is malformed and cannot be reliably restored, the extraction record is treated as invalid.

Reasoning-augmented extraction also changes RAG rendering. Examples retrieved for `Reasoned-RAG` use the same temporary output format expected from the current extractor. A direct example with final field values would be inconsistent with a prompt that asks for top-level `{reasoning, value}` objects. Reasoning-mode examples therefore demonstrate both the evidence protocol and the placement of the final answer in the `value` slot. The same labels, EL CAMPO PIDE, FRAGMENTOS RELEVANTES, and VALOR FINAL, appear inside example reasoning strings, so the model sees field interpretation, evidence selection, and final value placement while still treating examples as demonstrations rather than evidence for the current instance.

7. Postprocessing and Output Normalization

DRILLER uses deterministic postprocessing to align model outputs with the final schema interface. Even with a strict JSON Schema response format, outputs can contain representation artifacts such as literal Unicode escape strings or the string `"null"` where JSON `null` was intended. Postprocessing cleans these artifacts without changing the semantic content of the extraction.

The first step is JSON parsing. The submitted pipelines expect a JSON object because the prompt asks for JSON only and the request includes a structured response format. If an output cannot be

parsed as JSON, the trial is marked invalid rather than repaired by string heuristics. This boundary is especially important for `Mixed-SC`, where aggregation should combine valid candidate objects rather than fragments of malformed responses.

After parsing, DRILLER applies Unicode cleanup recursively to string values. Spanish text may include diacritics, the letter ñ, inverted punctuation marks, and other characters that can appear inconsistently across model outputs or API layers. The normalizer cleans literal escape artifacts such as `\u00e1` when they appear inside strings and then normalizes strings to Unicode Normalization Form C (NFC). NFC makes visually identical strings more stable for comparison while preserving their textual content.

The prompt asks the model to use JSON `null` for unsupported information when null is allowed, but models may emit string literals such as `"null"`, `"NULL"`, or other casing variants instead. DRILLER converts these string markers to JSON `null` only at schema positions where `null` is valid. It does not globally replace all occurrences, because a non-nullable string field should not be silently changed into a null value.

In `Mixed-SC`, each trial independently performs prompt construction, strict generation, parsing, Unicode cleanup, schema restoration when relevant, and null coercion. Invalid trials are excluded from aggregation. The aggregator receives postprocessed JSON candidates in the original schema shape; after aggregation, nullable-string coercion may be applied again so selected values remain aligned with schema-valid null representation.

8. Heterogeneous Self-Consistency

8.1. Motivation

Schema-guided extraction can vary across generations even when the output format is constrained. Candidate outputs may differ in whether they include a borderline entity, how they normalize a date, which enum label they choose, or whether they return JSON `null`. Some variance comes from sampling, and some comes from the interface shown to the model. The reasoning-augmented prompt, for example, spends more output budget on field-local evidence assessment before committing to values.

`Mixed-SC` thus uses heterogeneous self-consistency. Rather than sampling one prompt repeatedly, it combines `Schema-RAG`, which produces final values directly, with `Reasoned-RAG`, which temporarily produces `{reasoning, value}` objects and then unwraps them. This can expose different error patterns to the aggregator. The final decision is not a flat vote over complete JSON strings; it is a recursive schema-aware aggregation over candidate objects.

8.2. Candidate Generation

Let $q = (x, S)$ denote an extraction instance with source text x and target schema S. Let E_D be the direct extractor used by `Schema-RAG`, and let E_R be the reasoning-augmented extractor used by `Reasoned-RAG`. `Mixed-SC` defines the intended heterogeneous plan

$$\pi = (E_R, E_D, E_R, E_D).$$

The order is interleaved because execution is budget-aware. If the runner stops early, the available candidate set should not consist only of one extractor family. Interleaving exposes the aggregator to both direct and reasoning-augmented candidates as early as possible, so partial execution still preserves the heterogeneous premise of `Mixed-SC`. The plan starts with the reasoning-augmented extractor to prioritize the more evidence-sensitive configuration.

The runner executes an ordered prefix of this plan. If $k \leq 4$ trials fit under the effective token and time budgets, the completed trial indexes are $K = \{1, \ldots, k\}$. For each $i \in K$, the extractor π_i performs its own RAG retrieval, prompt construction, strict JSON generation, and deterministic postprocessing. Direct trials render examples and schemas in the `Schema-RAG` format. Reasoning-augmented trials render examples and schemas in the temporary `Reasoned-RAG` format, so demonstrations contain reasoning/value fields during generation.

Postprocessing maps each raw model response to either a final-shape candidate y_i or an invalid output. It applies Unicode normalization and schema-aware null coercion, and rejects malformed outputs. The aggregation input is therefore

$$Y = \{(i, y_i) : i \in K,\ \mathrm{valid}_S(y_i)\}.$$

The pair (i, y_i) preserves the trial identity used for support counting. The value y_i is always in the original challenge schema shape; raw model messages and reasoning traces are not aggregated. If $Y = \emptyset$, the system does not fabricate an extraction from partial information.

An incremental budget planner bounds trial execution. Before the first trial, the runner estimates how many planned attempts fit under the effective token and time budgets for the instance. After each completed extraction, it updates the estimate using observed prompt tokens when available, completion tokens, and elapsed trial time, then decides whether another planned extraction can still be attempted. Because the system does not fully control hosted-call token use, this continuation decision is made after completed trials rather than by assuming that all four planned calls will fit.

8.3. Schema-Aware Recursive Aggregation

Aggregation recurses over the target schema. Let $A_S(Y)$ denote the aggregation of indexed candidate values Y under schema S. Complete JSON objects are not treated as indivisible. If S is an object schema with fields F_S, each emitted field is aggregated from the values observed for that field:

$$Y_f = \{(i, y_i[f]) : (i, y_i) \in Y,\ y_i \text{ is an object},\ f \in y_i\},$$

$$A_S(Y)[f] = A_{S_f}(Y_f),$$

where S_f is the child schema for field f. Required root fields are always considered when present in the schema. Optional nested fields are emitted only when enough candidates contain the property, so a single stray optional key does not automatically enter the final object. Field-wise recursion allows one candidate to contribute stronger support for one field while another contributes a better-supported value elsewhere.

The recursion depends on a schema-aware similarity function $\mathrm{sim}_S(a, b) \in [0, 1]$. For exact scalar types, including enums, booleans, integers, and numeric values, agreement is canonical equality:

$$\mathrm{sim}_S(a, b) = \mathbb{1}[a = b].$$

For strings, sim_S uses the configured string comparator. The submitted default normalizes case and accents, then combines token overlap and character n-gram overlap. Minor spelling, accent, punctuation, or phrasing differences can cluster together, but the aggregator still selects a generated string.

For object values, similarity is computed field by field. Let $F(a, b) \subseteq F_S$ be the fields present in at least one of the two objects, and let w_f be a small schema-dependent field weight. Missing-on-one-side fields contribute zero similarity but still count in the denominator:

$$\mathrm{sim}_S(a, b) = \frac{\sum_{f \in F(a,b)} w_f \mathbb{1}[f \in a \wedge f \in b]\, \mathrm{sim}_{S_f}(a[f], b[f])}{\sum_{f \in F(a,b)} w_f}.$$

For arrays, similarity ignores exact position and compares items by a greedy one-to-one soft matching. If $M(a, b)$ is the greedy matching between items of arrays a and b, using the item-schema similarity $\mathrm{sim}_{S_{\mathrm{item}}}$, then

$$\mathrm{match}(a, b) = \sum_{(u,v) \in M(a,b)} \mathrm{sim}_{S_{\mathrm{item}}}(u, v),$$

$$\mathrm{sim}_S(a, b) = \frac{\mathrm{match}(a, b)}{|a| + |b| - \mathrm{match}(a, b)}.$$

Two empty arrays have similarity 1, while an empty and a non-empty array have similarity 0. This soft Jaccard-style score lets two arrays agree strongly when items appear in different orders or when object items differ only in secondary fields.

8.4. Clustering and Scalar Selection

For scalar and object positions, `Mixed-SC` first clusters candidate values. Let

$$V = \{(i, v_i) : (i, y_i) \in Y\}$$

be the indexed values observed at the current schema position. For exact types, clusters are canonical-equality groups. For non-exact types, candidates are processed in stable order and assigned to the compatible cluster with highest similarity, subject to two constraints: the similarity must reach the active threshold τ_S, and the cluster may contain at most one value from any trial. A cluster's support is therefore trial support, not occurrence count:

$$\mathrm{supp}(C) = |\{i : (i, v) \in C\}|.$$

Clusters are ranked by decreasing support, with stable first occurrence as a tie-breaker. Let $C^\star$ be the highest-ranked non-null cluster. For non-exact clusters, the selected representative is a medoid: a generated candidate value whose average schema-aware similarity to the cluster is maximal,

$$\mathrm{medoid}_S(C) = \arg\max_{x \in C} \frac{1}{|C|} \sum_{y \in C} \mathrm{sim}_S(x, y).$$

The aggregator therefore chooses among generated values rather than composing a new free-text answer. For object clusters, the representative is obtained by recursively aggregating the object fields of the cluster members.

Null values are handled separately. Let $n_\varnothing$ be the number of candidate values equal to JSON `null`. For nullable fields, `null` is selected only when its support is strictly greater than the best non-null support:

$$A_S(V) = \texttt{null} \quad \Longleftrightarrow \quad n_\varnothing > \mathrm{supp}(C^\star).$$

This tie policy avoids turning uncertainty into absence while still preserving shared abstention when multiple trials independently return `null`. If all candidates are null, the nullable field is returned as `null`; for non-nullable schemas, the aggregator falls back to the conservative empty value for the schema type.

8.5. Array Aggregation

Arrays require separate handling because disagreement can involve order, omissions, duplicates, merged items, split items, or outliers. A majority vote over complete arrays would be brittle: two candidates may contain the same substantive items in different orders or differ by one peripheral object. DRILLER therefore aggregates arrays by clustering items across candidate arrays.

For an array field, let A_i be the observed array from trial i. Items are first deduplicated within each A_i using the item-schema similarity and a high intra-trial threshold. This prevents repeated near-duplicates from one generated array from becoming multiple votes. The remaining indexed item candidates are then clustered across trials using $\mathrm{sim}_{S_{\mathrm{item}}}$, again with the restriction that each cluster can contain at most one item from a given trial.

Arrays of objects may use identity-like fields to improve alignment. Many object-array schemas contain a field that behaves like an identifier: a name, title, mention, text span, ingredient, symptom, reaction, or source fragment. When such a field h is selected with sufficient confidence, item clustering uses similarity on h to decide whether two objects refer to the same extracted entity. The fields inside

an aligned object cluster are aggregated recursively. If no useful identity field is detected, the aggregator falls back to broader object similarity.

This alignment matters because object arrays often combine extraction and classification. The identity-like field tells the system which candidate objects correspond; remaining fields can then be compared or aggregated inside that item cluster. Imperfect identity detection is also a risk: a generic attribute may align unrelated objects, while failure to detect an identity field may split corresponding objects into duplicate clusters.

8.6. Recall-Biased Selection

After item clustering, the aggregator builds a small set $\mathcal{Z}$ of candidate final arrays. One family of candidates uses support thresholds:

$$z_m = [\mathrm{rep}(C) : \mathrm{supp}(C) \geq m] \,,$$

where m ranges over configured support floors and support ratios, and $\mathrm{rep}(C)$ is the medoid or recursively aggregated representative of item cluster C. A second candidate is built greedily in a minimum Bayes risk (MBR) style: starting from the empty array, the builder adds the cluster whose representative most improves agreement with the observed arrays, stopping when no remaining cluster improves the score. [6]

For any candidate final array z, the MBR score is its average schema-aware agreement with the observed arrays:

$$R(z) = \frac{1}{|\mathcal{A}|} \sum_{A_i \in \mathcal{A}} \mathrm{sim}_{S_{\mathrm{array}}}(z, A_i),$$

where $\mathcal{A}$ is the set of non-null observed arrays for the field. Pure MBR would select the candidate with maximal $R(z)$. The submitted selector is recall-biased within a small tolerance. Let

$$R_{\max} = \max_{z \in \mathcal{Z}} R(z), \qquad \mathcal{E} = \{z \in \mathcal{Z} : R(z) \geq R_{\max} - \epsilon\}.$$

The final array is selected from $\mathcal{E}$ by preferring, in order, greater length, higher MBR score, and earlier candidate order:

$$z^\star = \arg\max_{z \in \mathcal{E}} \left(|z|,\ R(z),\ -\mathrm{order}(z)\right).$$

The default tolerance is $\epsilon = 0.06$, configurable at runtime. This favors retaining supported items when evidence from trials is close. The longer array is not accepted unconditionally; it must remain competitive under the consensus objective.

The design trades omission against over-extraction. A stricter selector would discard more borderline items, improving precision in some cases but risking missed entities or list elements. DRILLER's recall-biased selector keeps items with enough support to remain near the best consensus score, which is useful for unordered lists, object arrays, and evidence collections where candidates often agree on a core set but vary on peripheral supported items.

8.7. Conservativeness and Failure Modes

Aggregation is conservative: it selects among generated candidates rather than inventing semantic content. Exact scalar types are selected by canonical equality groups; non-exact string clusters choose a generated representative; arrays are built from generated item clusters. The aggregator does not use external knowledge, synthesize new enum labels, or create facts absent from all candidate outputs.

Self-consistency still does not guarantee correctness. If multiple trials share the same misunderstanding of a field description, attend to the same distractor mention, or overtrust an unsupported fact, aggregation can preserve that mistake with high support. Heterogeneous prompting reduces reliance

on one prompt representation, but all trials still share the source text, target schema, local FSP corpus, model endpoint, and broad extraction rules.

Array aggregation has additional failure modes. The recall-biased selector may retain extra borderline items, and identity-field detection can misalign object items or fail to merge true matches. Threshold choices also matter: loose thresholds can merge distinct values, while strict thresholds can split legitimate paraphrases or duplicate entities. These risks are the cost of trying to preserve recall in schemas where missing list members are common and where item order is not meaningful.

`Mixed-SC` also increases inference cost. It requests up to four model calls rather than one, and up to two calls use reasoning-augmented generation, which can produce longer intermediate outputs before unwrapping.

9. Experimental Setup

The submitted solution consists of three final pipelines: `Schema-RAG`, `Reasoned-RAG`, and `Mixed-SC`. These correspond to the submitted names `enriched-schema-rag`, `enriched-inline-reasoning-rag`, and `mixed-extractors-self-consistency-rag`.

Final evaluation follows the official GenSIE shared-task protocol. GenSIE evaluates Spanish extraction from a source text, an instruction, and a target JSON Schema, with systems returning grounded schema-conformant JSON objects. The GenSIE overview paper is the authoritative source for dataset construction, official scoring, ranking, model selection, and efficiency reporting [2].

DRILLER uses the hosted evaluation through an OpenAI-style chat interface. The submitted pipelines do not hard-code a private model identity: the evaluator supplies the model name at runtime, and the system forwards that model name to the hosted inference endpoint. Exact official model identifiers, decoding settings, evaluation split details, and token accounting should be taken from organizer-provided metadata or final run logs.

10. Results and Analysis

At the time of writing, official shared-task scores, ranks, per-model results, and final token-use figures have not been released in citable form. We therefore report no numeric results. Exploratory local artifacts are not used as final evidence because they are partial, based on local splits, or associated with non-final variants; mixing them with official results would make the comparison unclear.

Until official outputs are available, analysis is limited to qualitative error categories that should be quantified later by inspecting aligned gold and system outputs with the source text and schema. These categories are not claims about observed official frequencies.

Unsupported inference is a central risk. A model may return a value that is plausible or true in the world but absent from the current text. This is especially relevant for nullable fields and for fields resembling common factual questions. `Reasoned-RAG` makes evidence assessment explicit during generation, and `Schema-RAG` emphasizes grounded extraction rules, but final analysis should measure whether unsupported values remain a source of false positives.

Array errors form another likely category. Systems may miss list items, duplicate objects, extract only salient examples, or include distractor mentions. `Mixed-SC` can retain items generated by at least one valid trial through clustering and recall-biased selection, but aggregation cannot recover an item absent from every candidate and may preserve extra borderline items.

Nullability errors should be separated from other value errors. They include returning `null` when evidence exists, returning a value when absence is correct, and emitting the string `"null"` instead of JSON `null`. DRILLER postprocesses only the representational case when the schema permits null; the first two are semantic grounding errors.

Variable schemas can also induce confusion between similar fields, such as creator versus director, release year versus setting year, ingredient versus allergen, or legal title versus affected article. Schema-

readable prompting and field-aware retrieval are intended to reduce this risk, but final error analysis should check whether close field descriptions or distractor mentions still lead to swapped values.

Aggregation introduces its own trade-offs. `Mixed-SC` can preserve shared mistakes when several trials converge on the same wrong interpretation, and its recall-biased array selector may favor inclusion when candidate arrays are close in score. Official analysis should compare the three submitted pipelines by metric, cost, and error category without attributing component-level improvements unless supported by controlled ablations.

11. Limitations

Several limitations follow from DRILLER's schema-variable design. The first is dependence on a local few-shot prompting corpus. The demonstrations are synthetic and curated, which gives controlled coverage of nulls, populated values, enum choices, arrays, nested structures, and distractors, but also requires careful construction and review. Each case must keep the instruction, source text, schema, output, and rationale mutually consistent. Errors or narrow coverage in the corpus can propagate into prompts.

Coverage remains incomplete because GenSIE schemas vary by instance. A demonstration family can represent one schema well while failing to cover later fields with different names, descriptions, nesting, or value regimes. Complementary examples reduce reliance on one output pattern, but they cannot guarantee that every private evaluation schema has a close analogue. If retrieved examples mostly show populated arrays, frequent enum labels, or non-null values, they may still bias the model toward overproduction.

Retrieval depends on schema information quality. Same-schema retrieval is useful when structurally matching examples exist, but it has limited value when the local corpus lacks the target schema. Field-level fallback is more flexible, yet it relies on names, descriptions, type representations, and coarse type classes. Sparse or generic descriptions can make unrelated fields appear similar, while equivalent fields with different wording can be missed. Type compatibility prevents many implausible alignments, but it can also exclude examples whose meanings are related despite different schema forms.

The reasoning/value wrapper in `Reasoned-RAG` is a cost trade-off. It can encourage explicit evidence assessment and clearer decisions between a value and `null`, but it increases output length, consumes context, and may raise latency or token cost. Because the submitted wrapper is top-level only, it does not give every nested field its own explicit evidence scaffold. This keeps the temporary schema manageable, but complex nested objects may still require decisions that are only indirectly guided by top-level reasoning.

`Mixed-SC` further increases inference cost by requesting up to four trials, including up to two reasoning-augmented calls. Runtime budgets may limit completed trials, and practical efficiency depends on the model, backend, decoding configuration, and official token accounting. Its value must be judged jointly by extraction quality and cost.

Aggregation is heuristic. It votes over closed scalar values, clusters strings, aggregates objects field by field, clusters array items, and selects arrays with a recall-biased strategy. These rules are schema-aware and conservative in the sense that selected representatives come from generated candidates, but they are not optimality guarantees. Shared mistakes can be reinforced, borderline array items can be retained, and imperfect identity-field detection can misalign object arrays.

Component-level effects remain empirical questions. Claims about the separate impact of schema rendering, retrieval, same-schema prompting, reasoning wrappers, postprocessing, or heterogeneous self-consistency require controlled ablations. Results may also depend strongly on the hosted model's structured-output behavior, context limits, multilingual extraction ability, and instruction following.

12. Conclusion

DRILLER submitted a modular schema-guided extraction system for GenSIE 2026. It treats each instance as Spanish information extraction under a variable JSON Schema, where the central challenge is interpreting field semantics from the current instruction, schema, and source text while returning grounded schema-conformant JSON.

The three submitted pipelines are `Schema-RAG`, `Reasoned-RAG`, and `Mixed-SC`. All use retrieval-augmented prompting. The common extraction spine combines schema-readable Pydantic-style prompting, strict structured JSON generation, field-aware few-shot retrieval, and conservative postprocessing. `Reasoned-RAG` adds temporary top-level reasoning/value wrappers that are removed before final output. `Mixed-SC` combines direct and reasoning-augmented candidates through heterogeneous self-consistency and recursive schema-aware aggregation.

The main design principle is to treat the schema as a runtime semantic specification rather than only an output validator. Readable prompt schemas help the model interpret fields; strict generation constrains the response interface; retrieved examples illustrate field behavior; postprocessing normalizes representation-level artifacts without semantic repair; and aggregation reconciles candidate values according to schema type.

Future work should strengthen retrieval and aggregation. Learned retrieval could complement schema fingerprints and field-similarity heuristics, especially when field descriptions are sparse. Automatic validation of FSP cases would help detect inconsistent demonstrations before they enter prompts. Aggregation could benefit from better uncertainty estimates, learned item alignment, and schema-specific constraints. Controlled ablations are needed to measure component contributions, and robust array/object extraction remains a priority because omissions, duplicates, and field-alignment errors interact strongly in nested structures.

Declaration on Generative AI

During the preparation of this manuscript, generative AI tools assisted with repository inspection, organization of technical notes, drafting, and language editing. The authors reviewed, edited, and approved the final content and take full responsibility for the submitted paper. DRILLER also uses synthetic and curated few-shot resources as part of its retrieval component; an agentic LLM coding assistant helped prepare provisional resources, and the authors manually reviewed and validated them before use.

References

[1] Z. Wen, S. Liang, Y. Wu, Y. Zhang, Y. Liu, Effective and efficient schema-aware information extraction using on-device large language models, 2025. URL: https://arxiv.org/abs/2505.14992. arXiv:2505.14992.

[2] GenSIE Organizers, Overview of GenSIE 2026: Spanish schema-guided information extraction, in: Proceedings of IberLEF 2026, 2026. To appear.

[3] IberLEF Organizers, Overview of IberLEF 2026, in: Proceedings of IberLEF 2026, 2026. To appear.

[4] GenSIE Organizers, GenSIE 2026 submission guidelines, 2026. Challenge documentation.

[5] GenSIE Organizers, GenSIE 2026 task description, 2026. Challenge documentation.

[6] S. Kumar, W. Byrne, Minimum Bayes-risk decoding for statistical machine translation, in: Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004, Association for Computational Linguistics, Boston, Massachusetts, USA, 2004, pp. 169–176.